Informed Search

Alapan Kuila

IIITDM Kurnool

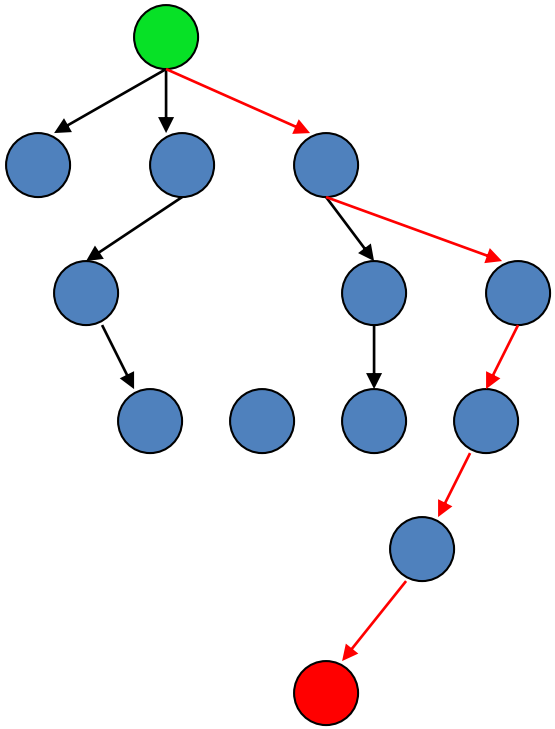
Artificial Intelligence

Slides adapted from materials by

Stuart Russell, Subbarao Kambhampati, Mausam, Dan Weld, Oren Etzioni, Henry Kautz, and Richard Korf

Informed (Heuristic) Search

Idea: be **smart** about what paths to try.



Blind Search vs. Informed Search

- What's the difference?

- How do we formally specify this?

  A node is selected for expansion based on an evaluation function that estimates cost to goal.

General Tree Search Paradigm

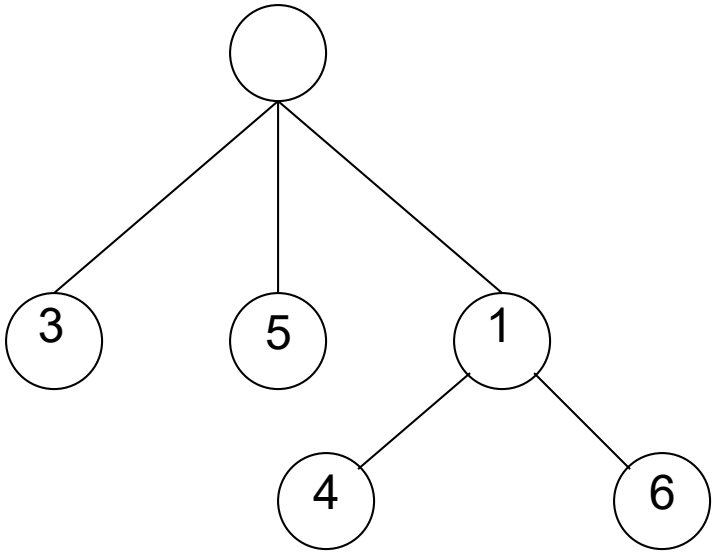
```
function tree-search(root-node)
   fringe ← successors(root-node)
   while ( notempty(fringe) )
         {node ← remove-first(fringe) //lowest f value
            state ← state(node)
            if goal-test(state) return solution(node)
            fringe ← insert-all(successors(node),fringe) }
   return failure
end tree-search
```

General Graph Search Paradigm

```
function tree-search(root-node)
   fringe ← successors(root-node)
   explored ← empty
   while ( notempty(fringe) )
         {node ← remove-first(fringe)
             state ← state(node)
             if goal-test(state) return solution(node)
             explored ← insert(node,explored)
             fringe ← insert-all(successors(node),fringe, if node not in explored)
         }
   return failure
end tree-search
```

Best-First Search

- Use an evaluation function f(n) for node n.
- Always choose the node from fringe that has the lowest f value.

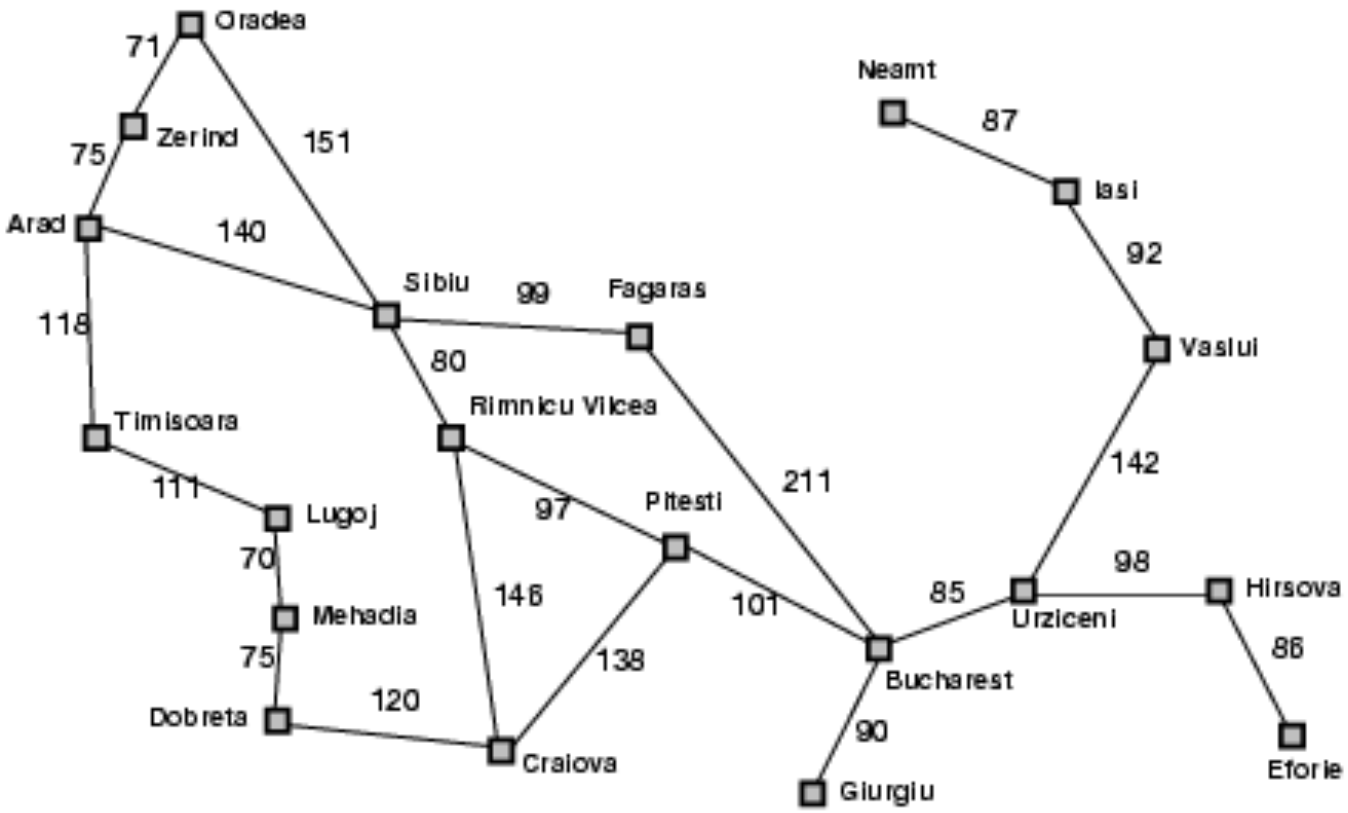


Best-first search

- A search strategy is defined by picking the order of node expansion
- Idea: use an evaluation function *f(n)* for each node
  - estimate of "desirability“

  → Expand most desirable unexpanded node
- Implementation:
  Order the nodes in fringe in decreasing order of desirability
- Special cases:
  - greedy best-first search
  - A* search

Romania with step costs in km



Old (Uninformed) Friends

- Breadth First =
  - Best First
  - with f(n) = depth(n)

- Uniform cost search =
  - Best First
  - with f(n) = the sum of edge costs from start to n

g(n)

Greedy best-first search

- Evaluation function $f(n) = h(n)$ (heuristic function)
  = estimate of cost from n to *goal*

- e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest

- Greedy best-first search expands the node that appears to be closest to goal

Properties of greedy best-first search

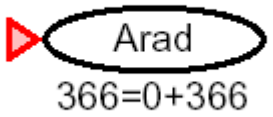
- Complete?
  - No – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →
- Time?
  - $O(b^m)$, but a good heuristic can give dramatic improvement
- Space?
  - $O(b^m)$ -- keeps all nodes in memory
- Optimal?
  - No

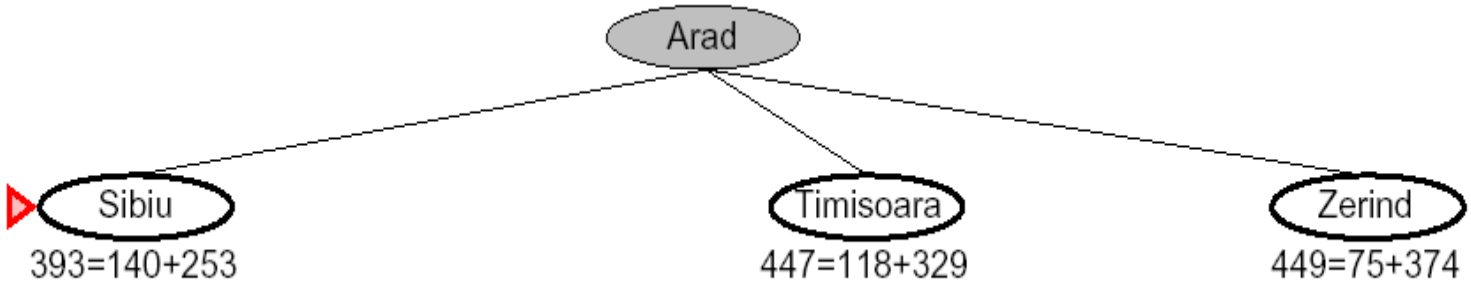
A* search

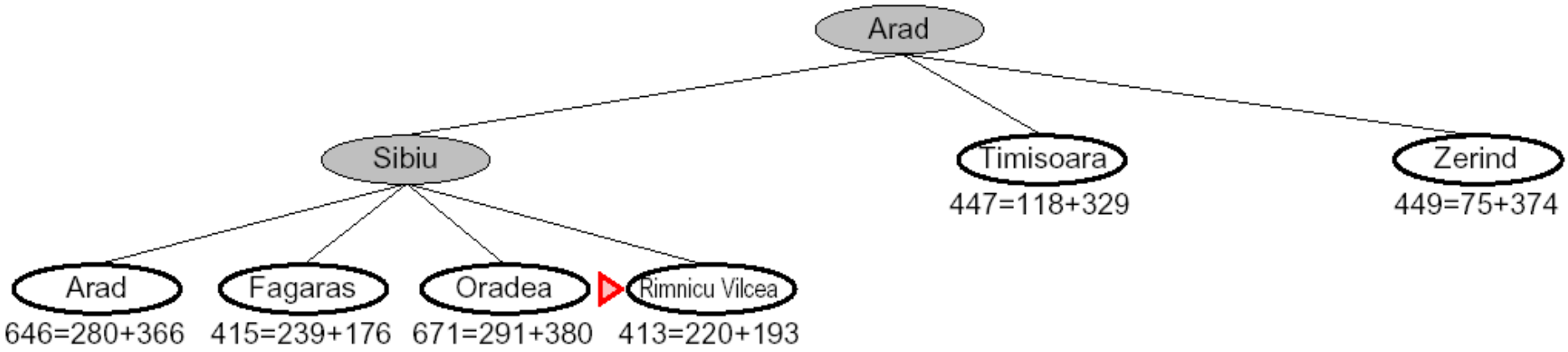
- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$

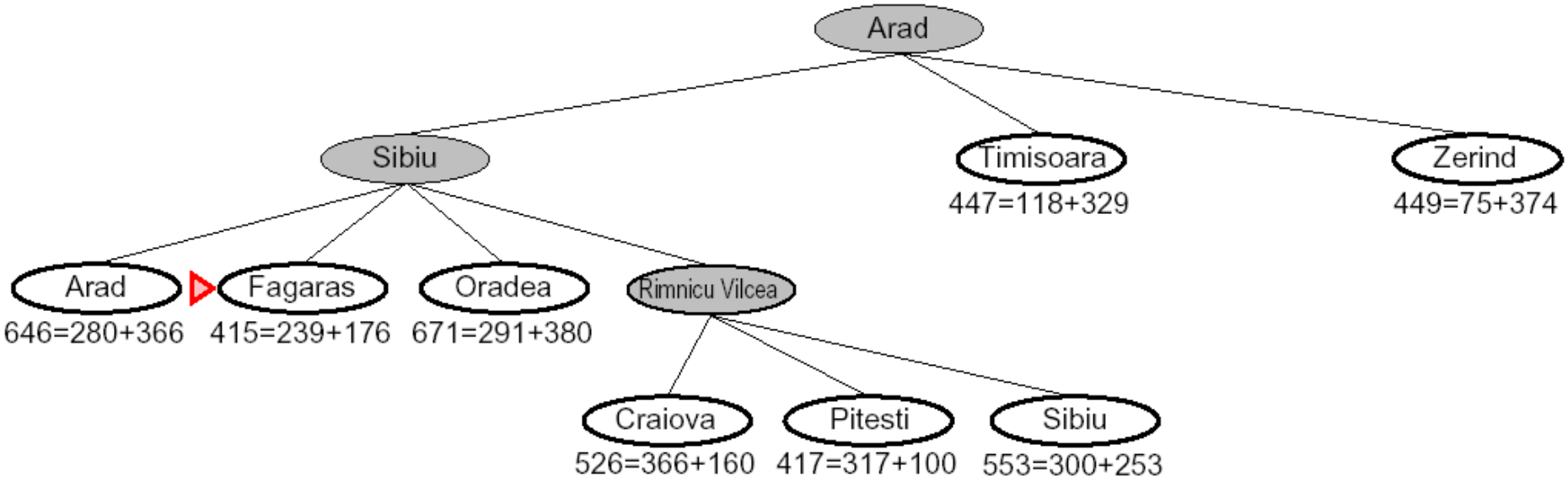
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal

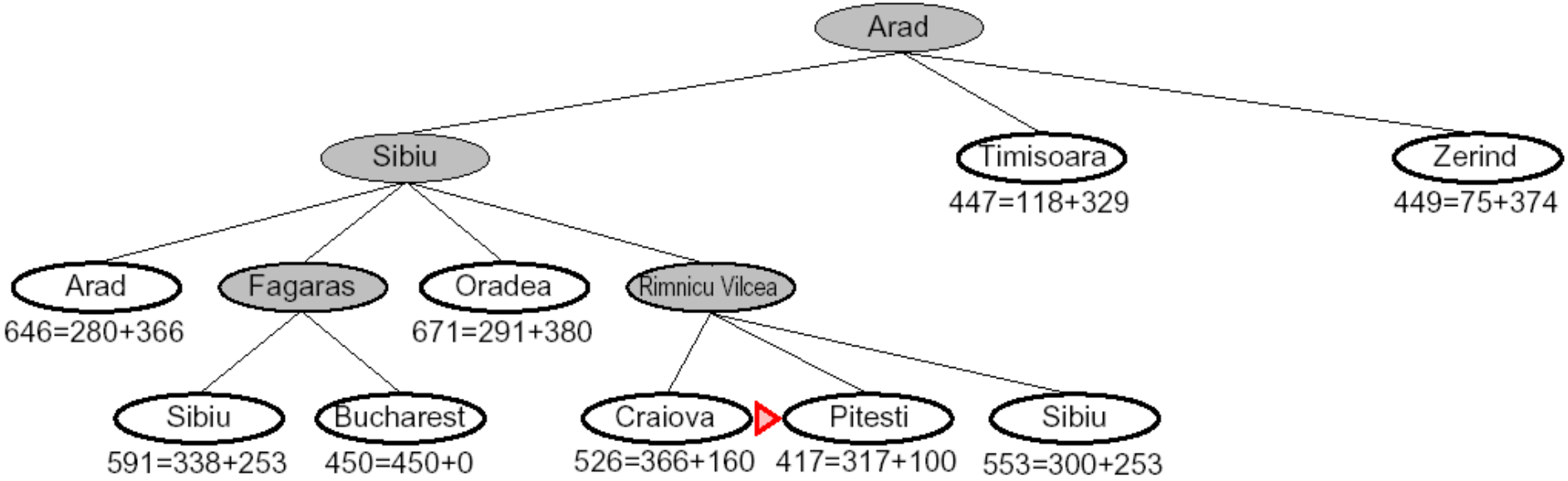
A* for Romanian Shortest Path

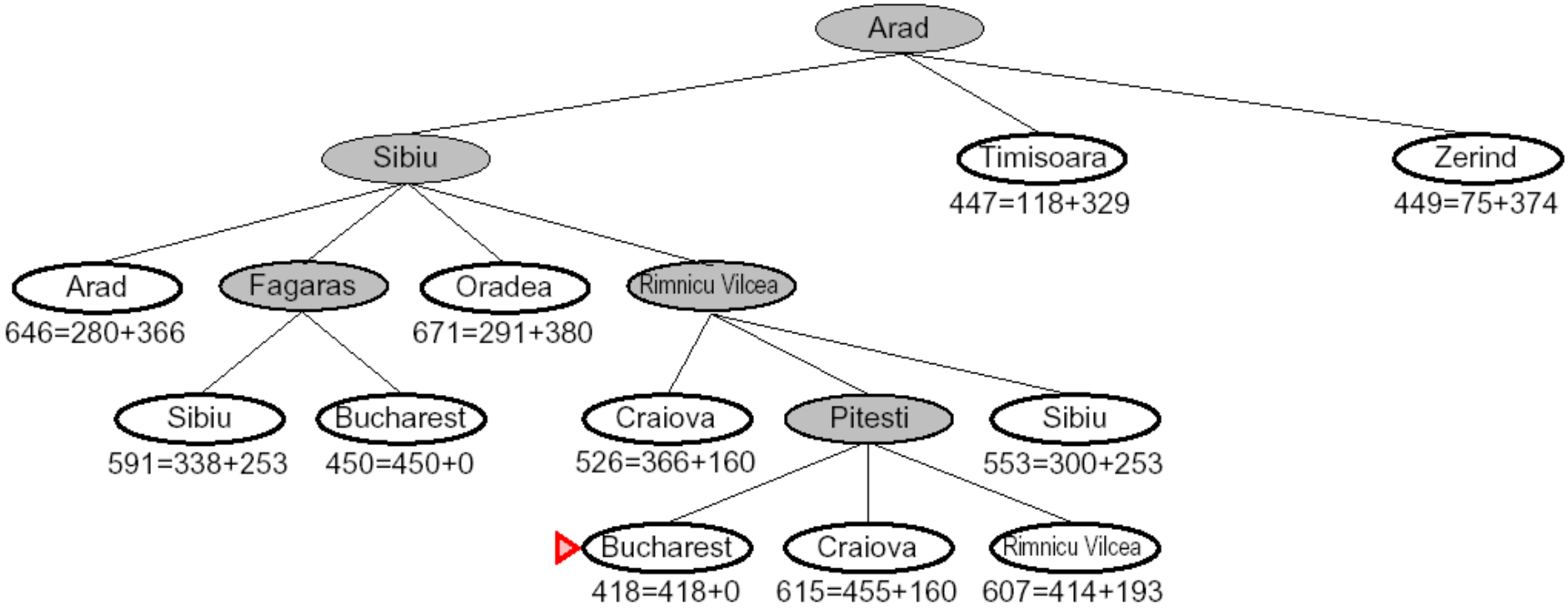



Arad
Sibiu
393=140+253
Timisoara
447=118+329
Zerind
449=75+374


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
413=220+193


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
415=239+176
Oradea
671=291+380
Rimnicu Vilcea
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
Sibiu
553=300+253
Bucharest
418=418+0
Craiova
615=455+160
Rimnicu Vilcea
607=414+193

Admissible heuristics

- A heuristic function $h(n)$ is admissible if for every node n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n.

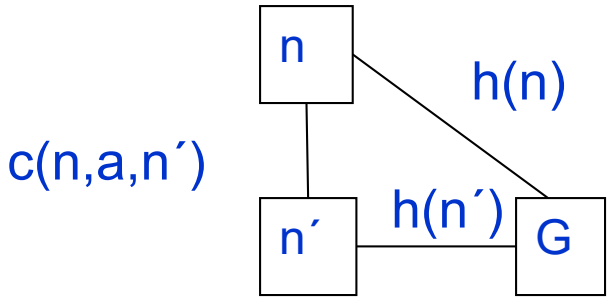
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic

- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)

- Theorem: If $h(n)$ is admissible, A* using `TREE-SEARCH` is optimal

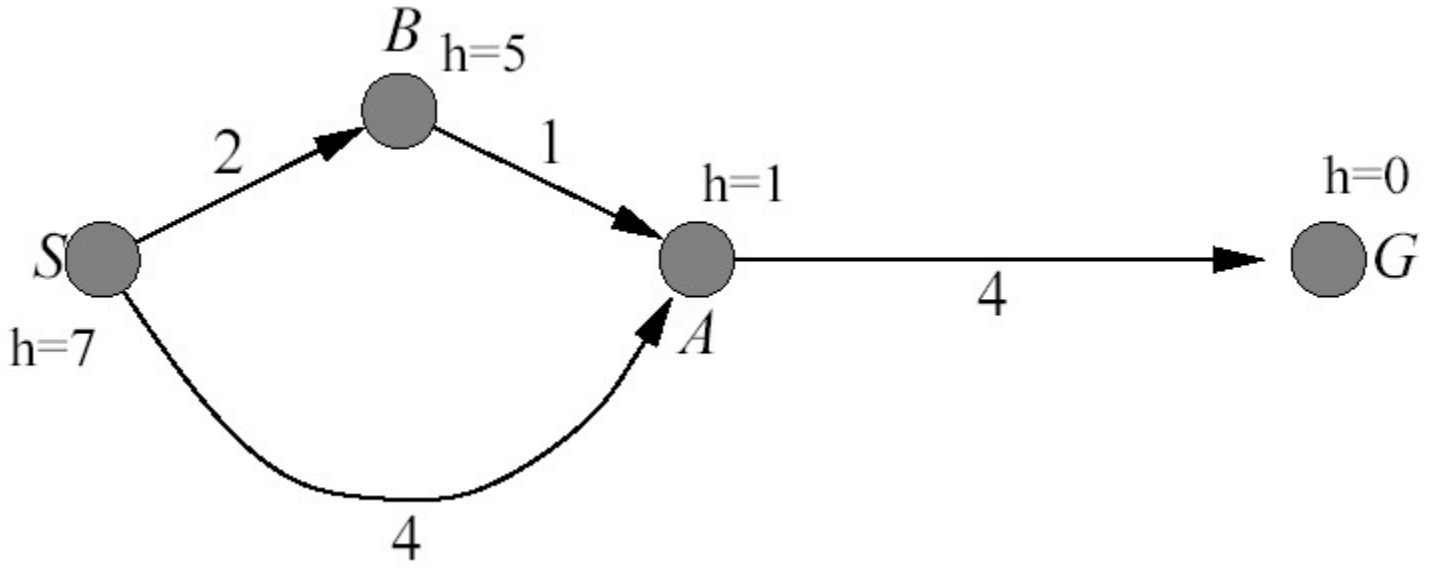
Consistent Heuristics

- h(n) is consistent if
  - for every node n
  - for every successor n´ due to legal action a
  - h(n) <= c(n,a,n´) + h(n´)



- Every consistent heuristic is also admissible.
- Theorem: If *h(n)* is consistent, A* using GRAPH–SEARCH is optimal

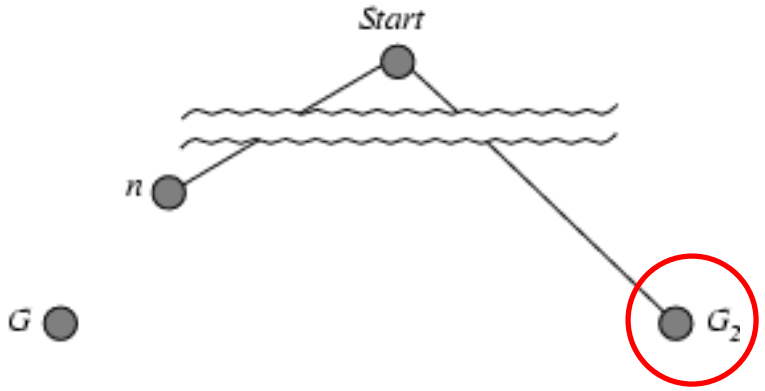
Example



Source: http://stackoverflow.com/questions/25823391/suboptimal-solution-given-by-a-search

Proof of Optimality of (Tree) A*

- Assume *h*() is admissible.
  Say some sub-optimal goal state G_2 has been generated and is on the frontier.
  Let *n* be an unexpanded state such that *n* is on an optimal path to the optimal goal *G*.



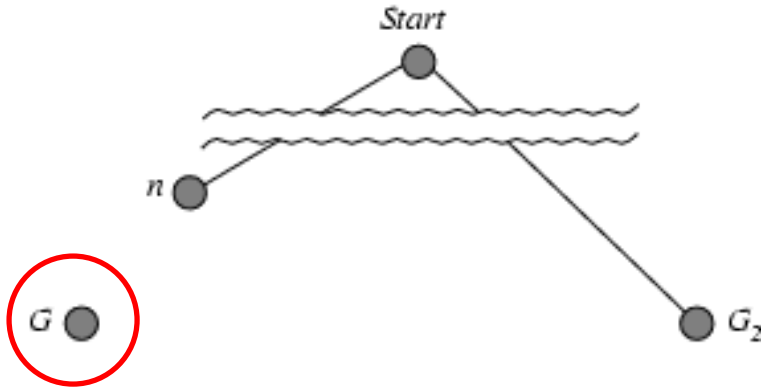
Focus on G_2:

$f(G_2) = g(G_2)$ since $h(G_2) = 0$

$g(G_2) > g(G)$ since G_2 is suboptimal

Proof of Optimality of (Tree) A*

- Assume *h*() is admissible.
  Say some sub-optimal goal state G_2 has been generated and is on the frontier.
  Let *n* be an unexpanded state such that *n* is on an optimal path to the optimal goal *G*.



$f(G_2) = g(G_2)$ since $h(G_2) = 0$

$g(G_2) > g(G)$ since G_2 is suboptimal

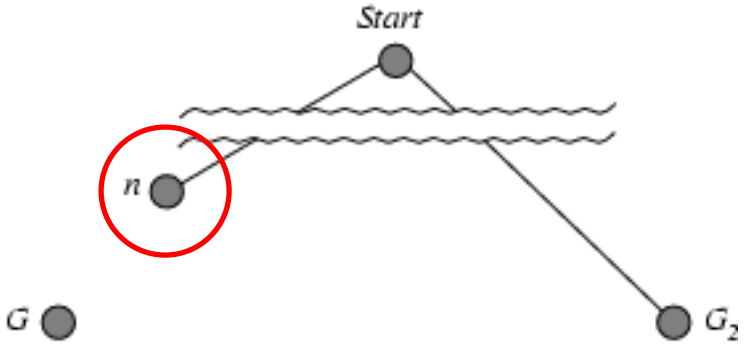
Focus on G:

$f(G) = g(G)$ since $h(G) = 0$

$f(G_2) > f(G)$ substitution

Proof of Optimality of (Tree)A*

- Assume *h*() is admissible.
  Say some sub-optimal goal state G_2 has been generated and is on the frontier.
  Let *n* be an unexpanded state such that *n* is on an optimal path to the optimal goal *G*.



$f(G_2) = g(G_2)$ since $h(G_2) = 0$
$g(G_2) > g(G)$ since G_2 is suboptimal

$f(G) = g(G)$ since $h(G) = 0$
$f(G_2) > f(G)$ substitution

Now focus on n:

$h(n) \le h^*(n)$ since h is admissible
$g(n) + h(n) \le g(n) + h^*(n)$ algebra
$f(n) = g(n) + h(n)$ definition
$f(G) = g(n) + h^*(n)$ by assumption
$f(n) \le f(G)$ substitution

Hence $f(G_2) > f(n)$, and A^* will never select G_2 for expansion.

Properties of A*

- Complete?

  Yes (unless there are infinitely many nodes with f $\leq f(G)$)

- Time? Exponential (worst case all nodes are added)

- Space? Keeps all nodes in memory

- Optimal?

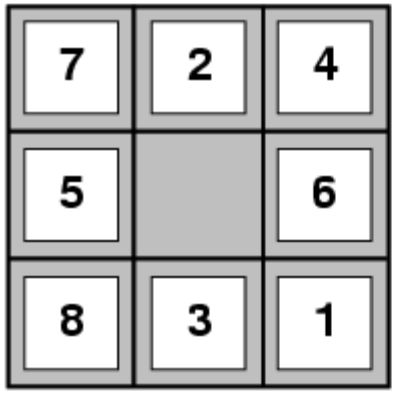
  Yes (depending upon search algo and heuristic property)

Admissible heuristics

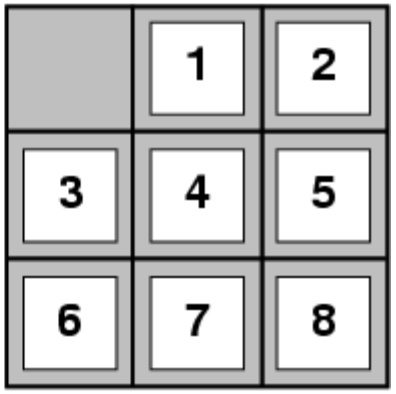
E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)



Start State



Goal State

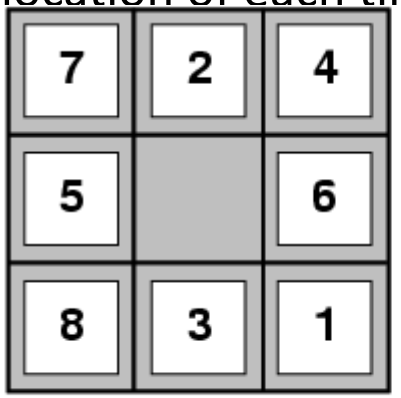
- $h_1(S) = ?$
- $h_2(S) = ?$

Admissible heuristics

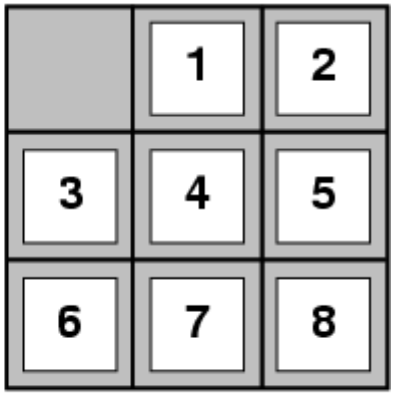
E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)



Start State



Goal State

- $h_1(S) = ?$ 8
- $h_2(S) = ?$ 3+1+2+2+2+3+3+2 = 18